

Final Pass Down Backend

Laravel 10 API backend for Final Pass Down, a digital legacy and life-record platform. The application lets a parent/main account store sensitive personal records, final wishes, documents, family contacts, subscriptions, memories, and emergency information, then grant controlled access to contacts such as guardians, legacy contacts, emergency contacts, and pet emergency contacts.

The backend is API-first and is consumed by mobile/web clients through `/api/*` endpoints. Authentication uses Laravel Sanctum tokens.

Tech Stack

- PHP 8.1+
- Laravel 10
- MySQL or MariaDB
- Laravel Sanctum for API authentication
- Stripe for subscription and customer/payment flows
- Firebase for push notifications
- Twilio for SMS verification codes
- DomPDF for PDF generation
- Laravel scheduler for reminders and subscription jobs

Project Structure

Important directories:

- `app/Http/Controllers/Api` - user-facing API controllers.
- `app/Http/Controllers/Admin` - admin panel API controllers.
- `app/Services` - business logic for each module.
- `app/Models` - Eloquent models and relationships.
- `app/Http/Resources` - API response transformers.
- `app/Http/Middleware` - role, passcode, guardian, and subscription checks.
- `app/Helpers` - global helper functions such as account owner resolution and upload helpers.
- `database/migrations` - table definitions.

- database/seeder - required seed data such as folders, questionnaire questions, plans, and pages.
- routes/api.php - all admin, auth, and user API routes.

The common pattern is:

```
Route -> Controller -> Service -> Model -> Resource -> JSON response
```

Most module controllers stay thin and delegate work to their matching service class.

Local Setup

Requirements

Install the following before running the app:

- PHP 8.1 or higher
- Composer
- MySQL or MariaDB
- XAMPP, Laragon, Laravel Valet, or another local PHP stack
- Node is not required for the API itself unless frontend assets are added later

Installation

```
composer install
cp .env.example .env
php artisan key:generate
```

Configure the database in .env:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=final_pass_down
DB_USERNAME=root
DB_PASSWORD=
```

Run migrations and seeders:

```
php artisan migrate
php artisan db:seed --class=FolderSeeder
php artisan db:seed --class=QuestionnaireSeeder
php artisan db:seed --class=FuneralPlanningQuestionsSeeder
php artisan db:seed --class=PageSeeder
php artisan db:seed --class=StorageSubscriptionPlanSeeder
```

Create the public storage symlink:

```
php artisan storage:link
```

Start the API:

```
php artisan serve
```

For XAMPP, point your web server to the project and ensure `public/index.php` is the web entry point.

Environment Configuration

`.env.example` is minimal. Production and staging environments also need values for external services.

App

```
APP_NAME="Final Pass Down"  
APP_ENV=local  
APP_DEBUG=true  
APP_URL=http://localhost/final-pass-down-backend
```

Mail

Used for welcome emails, password reset, OTP/login codes, and contact notifications.

```
MAIL_MAILER=smt  
MAIL_HOST=  
MAIL_PORT=  
MAIL_USERNAME=  
MAIL_PASSWORD=  
MAIL_ENCRYPTION=  
MAIL_FROM_ADDRESS=  
MAIL_FROM_NAME="${APP_NAME}"
```

Stripe

Used by subscription/customer services.

```
STRIPE_PUBLISHABLE_KEY=  
STRIPE_SECRET_KEY=
```

Twilio

Used by SMS verification and login-code flows.

```
TWILIO_SID=  
TWILIO_AUTH_TOKEN=  
TWILIO_PHONE_NUMBER=
```

Firebase

Used for push notifications.

```
FIREBASE_PROJECT=  
FIREBASE_CREDENTIALS=/absolute/path/to/firebase_credentials.json  
FIREBASE_DATABASE_URL=  
FIREBASE_STORAGE_DEFAULT_BUCKET=
```

Do not commit real Firebase credential JSON files or API secrets.

API Base Paths

Public API root:

```
/api
```

Main route groups:

- /api/user/* - mobile/web user APIs.
- /api/admin/* - admin APIs.
- /api/country/* - address lookup APIs.
- /api/reset/password, /api/verify/code, /api/set/password - shared password reset flow.

Protected user routes use:

```
auth:sanctum  
user  
require.access.code
```

Most user content write routes also pass through:

```
guardian.readonly  
subscription.storage
```

Authentication Flow

Signup

Endpoint:

```
POST /api/user/signup
```

The signup flow:

1. Creates the user.

2. Creates a Stripe customer.
3. Marks email as verified in the current backend flow.
4. Creates a self `user_contacts` row with type `user`.
5. Sends user/admin notifications.
6. Returns the user resource.

Login

Endpoint:

```
POST /api/user/login
```

The login flow supports email or phone login. It validates:

- Credentials.
- User status.
- Contact role restrictions.
- Two-factor email/SMS settings.
- Passcode/access-code state.
- Device token updates.

Successful login returns a Sanctum bearer token.

Use this header for protected requests:

```
Authorization: Bearer <token>  
Accept: application/json
```

Passcode And Inactivity

Important endpoints:

- `POST /api/user/verify/pass/code`
- `POST /api/user/set/pass/code`
- `POST /api/user/passcode/enable/update`

The `require.access.code` middleware checks whether a user must re-enter an access code after inactivity.

Login As Contact

Endpoint:

```
POST /api/user/login-as
```

This activates a contact role through `user_contacts.logged_in_by`. Many modules resolve the real data owner using `activeRole`, `created_by`, or the authenticated user id.

Account And Role Flow

The app has one main account owner and several contact roles.

Primary user/contact types:

- user
- Legacy Contact
- Guardian Contact
- Emergency Contact
- Pet Emergency Contact

Contacts are stored as normal users records and linked to the parent through `user_contacts`:

- `user_contacts.user_id` - parent/main account id.
- `user_contacts.contact_id` - contact user's id.
- `user_contacts.type` - role.
- `user_contacts.slug` - unique parent/contact/type identifier.
- `user_contacts.logged_in_by` - marks active role switching.

Parent-owned data should be read through the account owner id, not always `auth()->id()`. The helper `resolveAccountOwnerUserId()` handles:

1. `activeRole->user_id` when the user is acting as a contact.
2. `created_by` when a contact logs in normally.
3. The authenticated user's own id for parent accounts.

This is important for guardian and legacy access because their data usually belongs to the parent.

Guardian Access Flow

Guardians are read-only contacts. Their access is controlled by folder permissions.

Main pieces:

- `GuardianContactService` creates guardian users and user-contact rows.

- FolderAccess stores which folders/subfolders a guardian may access.
- RestrictGuardianContact blocks guardian write actions where required.
- FolderController and AuthController@homeV1 return only allowed folders for guardians.

When the guardian logs in, module APIs should resolve the parent account and return only data that belongs to the parent and is permitted by folder access.

Legacy Contact Flow

Legacy contacts can be configured with:

- Valid ID uploads.
- Login/security questions.
- Death certificate approval flow.
- Parent data access after approval.

Related modules:

- LegacyContactService
- UserLoginQuestionService
- DeathCertificateService
- RestrictLegacyContactActions

Emergency Contact Flow

Emergency contacts are stored through user_contacts and user records.

Endpoint:

```
GET /api/user/emergency-contacts
```

The response includes:

- Emergency contacts.
- Pet emergency contacts.
- Friends & Family records created by the parent, mapped into the emergency contact response shape.

Friends & Family records come from:

```
POST /api/user/friend/families
```

Home API Flow

Primary endpoint:

```
GET /api/user/v1/home
```

This endpoint returns:

- User profile data.
- Memories.
- Legacy contact.
- Emergency users.
- Guardian user.
- Allowed folders/modules.
- Files.
- Active role.

Parent users receive their normal module folder list. Guardians receive only folders granted through `folder_accesses`, including allowed subfolders.

Folder And Module Flow

Folders are seeded by `FolderSeeder` and define the mobile/web navigation modules.

Key fields:

- name
- icon
- route
- web_route
- child_route
- parent_id
- priority
- restricted

Main modules include:

- Friends & Family
- Pet Records
- Digital Diary
- Vital Clone
- Messages For Loved Ones

- Digital File Cabinet
- Memories
- Taxes
- Portfolio Management
- My Final Wishes
- Collectible & Keepsake
- Calendar Reminder
- Password Manager
- AutoPay & Subscription
- Emergency Info

The Digital File Cabinet contains submodules such as:

- Day Care
- Insurance
- Kids Activities
- Job History
- Real Estate
- Warranties
- Things I Help Others With
- Secrets And Safe Keeping
- ID Keeper
- Goals & Bucket List
- Favorite Places
- Weapons Locker
- Awards & Achievement
- Utilities
- Personal Folder
- License & Permits
- Vehicles
- Miscellaneous
- Travel Planner

Main User APIs

Most endpoints are under `/api/user` and follow these patterns:

```
GET    /api/user/<module>
POST   /api/user/<module>
GET    /api/user/<module>/{id}
POST   /api/user/<module>/{id}/update
DELETE /api/user/<module>/{id}
```

Important groups:

- Auth/profile: home, v1/home, profile, profile/update, change-password.
- Contacts: friend/families, legacy-contacts, guardian-contacts, emergency-contacts.
- Folder access: folders, folder, folder-access.
- Life records: pet-records, memories, messages, reminders, occasions.
- Final wishes: questionnaires, questionnaire-answers, funeral-plannings, final-wish-or-will-testaments.
- Emergency info: emergency-info, emergency-info-profile, allergies, dietary-or-medications.
- Digital file cabinet: day-cares, insurances, kids-activities, job-informations, real-estates, warranties, utilities, id-keepers, license-permits, vehicles, miscellaneous-documents, personal-folders, personal-folder-media.
- Financial records: portfolios, portfolio-personal-accounts, portfolio-business-accounts, retirements, taxes, draft-autopay-subscription, password-managers.
- Personal legacy: collectibles-keepsakes, goals-or-buckets, personal-yelps, weapons, awards-achievements, travel-planners, secret-and-safe-keepings.

See `routes/api.php` for the exact route names and controller mappings.

Final Wishes And Questionnaire Flow

Questionnaire questions are seeded by `QuestionnaireSeeder`.

Endpoints:

```
GET    /api/user/questionnaires
GET    /api/user/questionnaire-answers?type=wish
POST   /api/user/questionnaire-answer
```

Answers are saved in `questionnaire_answers` against the account owner's user id. Parent users read/write their own answers. Guardians resolve to the parent account so they can read the parent's final wishes when they have access.

Related modules:

- `QuestionnaireService`
- `QuestionnaireAnswerService`
- `FinalWishOrWillTestamentService`

- FuneralPlanningService

Subscription And Storage Flow

The app has storage subscription plans and logs for access and storage limits.

Important endpoints include:

- Storage plan listing.
- Active subscription.
- Subscription history.
- Payment intent creation.
- Confirm payment.
- Free plan subscription.
- Cancellation.

The `subscription.storage` middleware applies to write methods:

```
POST  
PUT  
PATCH
```

It resolves the subscription owner, checks active subscription status, and compares used file storage with the plan limit.

Storage usage is calculated from `files.size_kb` across many modules, including memories, pet records, taxes, insurance, real estate, portfolios, personal folders, travel planner, documents, and more.

Scheduled subscription jobs run from `app/Console/Kernel.php`:

```
php artisan schedule:work
```

Scheduled commands:

- Send reminder notifications every minute.
- Process trial billing daily.
- Process recurring billing daily.
- Send premium access notifications daily.
- Send trial/grace notifications daily.
- Process data retention daily.

File Upload Flow

Uploads use the local/public filesystem.

Common request fields:

- `image` - single image.
- `images[]` - multiple images.
- `files[]` - documents or module files.
- `valid_id[]` - identity files for contacts.
- `delete_media_ids[]` - remove selected media on update.

Common storage patterns:

- `App\Models\File` for polymorphic document/file uploads.
- `App\Models\Image` for polymorphic image uploads.
- `FileUploadTrait` and helper functions store files and generate URLs.

Generated URLs rely on:

```
php artisan storage:link
```

Admin Flow

Admin endpoints live under:

```
/api/admin
```

Admin features include:

- Login/logout/profile.
- Dashboard stats and graphs.
- User listing and status management.
- Legacy/guardian details.
- Reports.
- Feedback and pages.
- Push notifications.
- Subscription plans.
- Storage subscription plans.
- Storage subscription logs.
- Marketplace/store/order modules.

Admin routes use `auth:sanctum` and `admin` middleware after login.

Response Format

Most API responses are returned through `JsonResponseService` and Laravel resources. Successful responses follow this shape:

```
{
  "data": {},
  "status": true,
  "execution_time": "0.0000"
}
```

Failure responses follow this shape:

```
{
  "data": {
    "message": {
      "failed": "Error message"
    }
  },
  "status": false,
  "execution_time": "0.0000"
}
```

Development Commands

Run the app:

```
php artisan serve
```

Run tests:

```
php artisan test
```

Run PHPUnit directly:

```
vendor/bin/phpunit
```

Clear caches:

```
php artisan optimize:clear
```

Regenerate autoload files after helper changes:

```
composer dump-autoload
```

Run scheduled jobs locally:

```
php artisan schedule:work
```

Run queue worker if queues are enabled:

```
php artisan queue:work
```

Seeding Notes

The application depends on seeded navigation and reference data. At minimum, seed:

- FolderSeeder
- QuestionnaireSeeder
- FuneralPlanningQuestionsSeeder
- PageSeeder
- StorageSubscriptionPlanSeeder

Use additional seeders as needed for development data.

Security Notes

- Never commit .env files.
- Never commit real Firebase credential JSON files.
- Never commit Stripe, Twilio, GitHub, or mail credentials.
- Review route comments and config files before deployment for accidental secret strings.
- Use HTTPS in production.
- Keep APP_DEBUG=false in production.
- Ensure APP_KEY is generated per environment.

Production Checklist

Before deploying:

1. Set APP_ENV=production and APP_DEBUG=false.
2. Configure production database credentials.
3. Configure mail, Stripe, Twilio, and Firebase credentials.
4. Run `composer install --no-dev --optimize-autoloader`.
5. Run migrations.
6. Run required seeders.
7. Run `php artisan storage:link`.
8. Run `php artisan config:cache`.
9. Run `php artisan route:cache`.
10. Configure scheduler cron:

```
* * * * * cd /path/to/final-pass-down-backend && php artisan schedule:run >> /dev/null 2>&1
```

11. Configure queue workers if QUEUE_CONNECTION is not sync.

12. Confirm public file URLs are working.
13. Confirm payment and notification integrations in the target environment.

Important Files

- routes/api.php - all API routes.
- app/Http/Controllers/Api/AuthController.php - user auth, home, profile, login-as.
- app/Services/Common/UserService.php - shared user auth/business logic.
- app/Models/User.php - user model, roles, contact relationships.
- app/Models/UserContact.php - parent/contact pivot.
- app/Helpers/CustomHelper.php - helper functions and account owner resolution.
- app/Http/Middleware/CheckSubscriptionStorage.php - storage subscription enforcement.
- app/Http/Middleware/RestrictGuardianContact.php - guardian read-only behavior.
- app/Services/GuardianContactService.php - guardian creation and folder access.
- app/Services/LegacyContactService.php - legacy contact creation and access.
- app/Services/EmergencyContactService.php - emergency contacts and Friends & Family merging.
- app/Services/StorageSubscriptionService.php - storage subscription lifecycle.
- database/seeder/FolderSeeder.php - application module navigation.